

Projekt správy assetů DS II

Adam Cvikl (CVI0014)

Asset Manager - správa assetů pro herní vývoj.

19.04.2026

Obsah

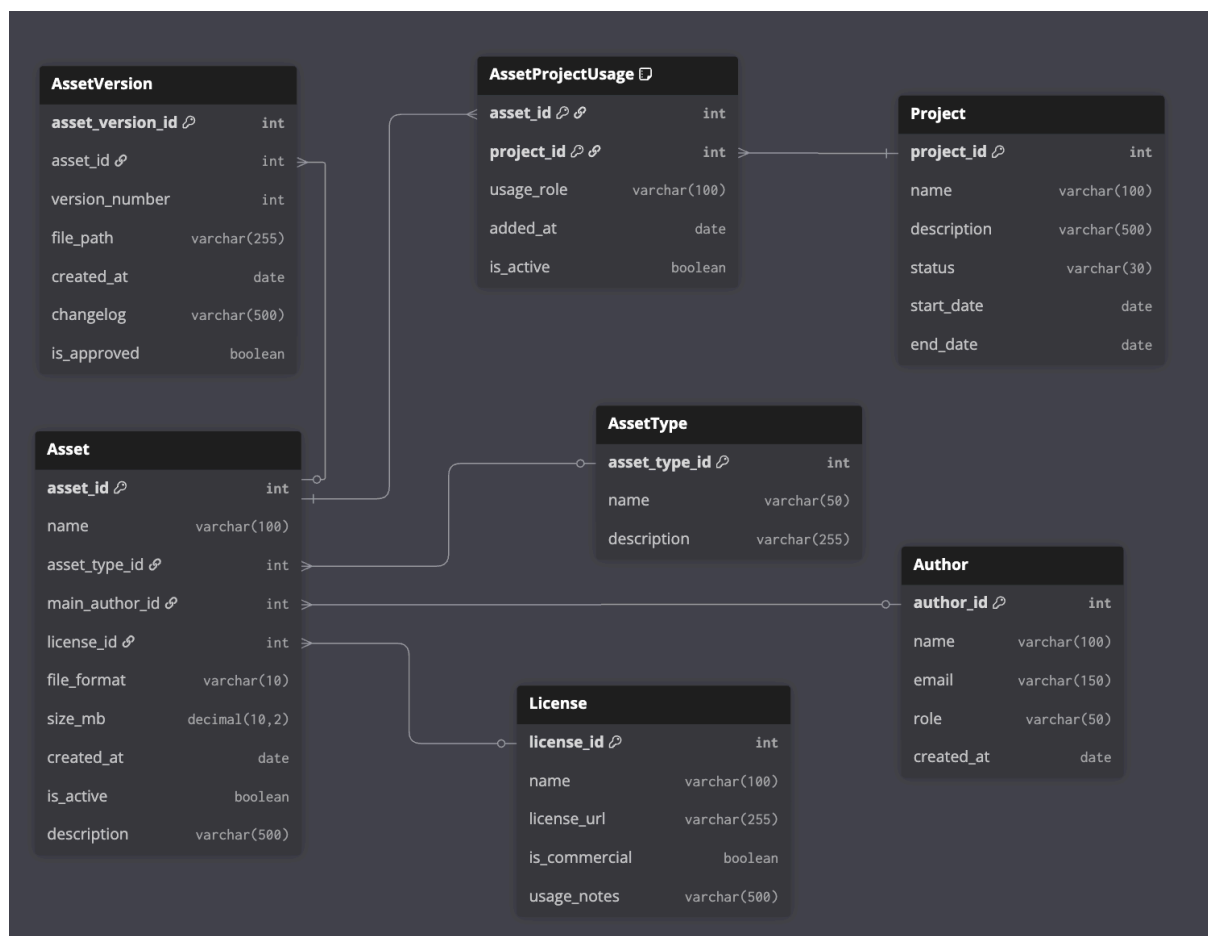
1. Specifikace programu	3
2. Relační datový model	4
3. Formulář	5
4. Seznam funkcí	7
4.1. CRUD F1 GetAuthor(p_author_id)	7
4.2. CRUD F2 GetAsset(p_asset_id)	7
4.3. F3 GetAssetVersions(p_asset_id)	7
4.4. F4 GetAssetProjects(p_asset_id)	7
4.5. F5 AddAsset(p_asset_id, p_name, p_asset_type_id, p_main_author_id, p_license_id, p_file_format, p_size_mb, p_created_at, p_is_active, p_description)	7
4.6. T1 F6 AddAssetVersion(p_asset_version_id, p_asset_id, p_version_number, p_file_path, p_created_at, p_changelog, p_is_approved)	8
4.7. T2 F7 AddAssetProjectUsage(p_asset_id, p_project_id, p_usage_role, p_added_at, p_is_active)	8
4.8. CRUD F8 EditAsset(p_asset_id, p_name, p_license_id, p_file_format, p_size_mb, p_is_active, p_description)	9
4.9. CRUD F9 FindAssets(p_asset_type_id, p_is_commercial)	9
4.10. F10 GetStats()	9
5. Popis tabulek databáze	10
5.1. Přehled tabulek	10
5.2. Tabulka Author	10
5.3. Tabulka AssetType	10
5.4. Tabulka License	10
5.5. Tabulka Project	11
5.6. Tabulka Asset	11
5.7. Tabulka AssetVersion	12
5.8. Tabulka AssetProjectUsage	12
6. Minispecifikace netriviální funkce	13
6.1. Vybraná transakce	13
6.2. Cíle transakce	13
6.3. Vstupy a výstupy	13
6.4. Minispecifikace krok za krokem	13
6.5. Poznámky k transakci	15
7. Závěr	16
Index of Figures	17

1. Specifikace programu

Primární úlohou programu je evidovat digitální assety používané při vývoji her a spravovat jejich životní cyklus od prvního vytvoření až po nasazení do konkrétního projektu. Systém umožňuje evidovat autory assetů, jejich typy, licenční podmínky, samotné assety, jednotlivé verze assetů a jejich použití v herních projektech.

U každého assetu se evidují základní informace, hlavní autor, licence, formát souboru, velikost, datum založení a aktivní stav. Součástí systému je také správa schválených verzí assetů a přehled projektů, ve kterých je asset použit. Důležitou funkcí je i filtrování assetů podle typu a licenčních omezení a zjištění, zda je možné daný asset bezpečně použít v konkrétním projektu.

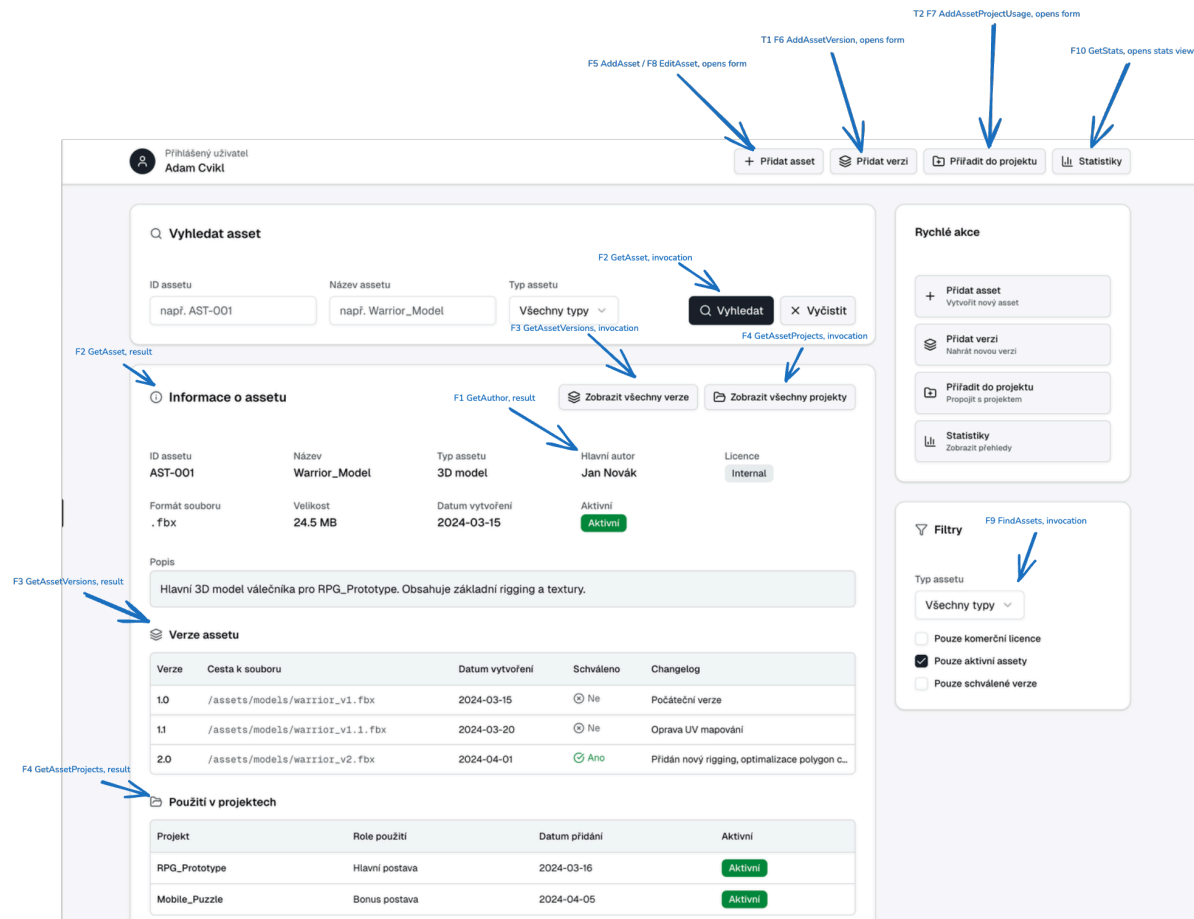
2. Relační datový model



Obrázek 1: Relační datový model systému správy assetů.

3. Formulář

Hlavní obrazovka aplikace vychází stejnou logikou z referenčního DS II dokumentu: vlevo je vyhledání a detail záznamu, pod ním formulář pro vložení nebo úpravu a v samostatném panelu se provádějí související akce nad verzemi a použitím assetu v projektech.



Obrázek 2: Vygenerovaný návrh hlavní obrazovky systému správy assetů.

ČÁST FORMULÁŘE	ÚČEL
Vyhledat asset	Vyhledání assetu podle identifikátoru nebo názvu a načtení jeho detailu.
Informace o assetu	Zobrazení názvu assetu, typu, autora, licence, formátu, velikosti, aktivity a popisu.
Zobrazit verze	Přehled všech verzí vybraného assetu včetně informace, která verze je schválena.
Zobrazit projekty	Seznam projektů, ve kterých je asset právě používán, včetně role použití.
Přidat / upravit asset	Formulář pro vložení nového assetu nebo úpravu jeho základních údajů.
Přidat verzi	Vložení nové verze assetu včetně cesty k souboru, changelogu a příznaku schválení.

Přiřadit do projektu	Zapsání použití assetu do konkrétního projektu s určením role a aktivního stavu.
Statistiky	Přehled nejčastěji používaných assetů v projektech.

4. Seznam funkcí

4.1. CRUD F1 `GetAuthor(p_author_id)`

Funkce vrátí jméno autora. Vstupním parametrem je identifikátor autora, podle kterého vrátí výsledek.

```
select name from Author
where author_id = p_author_id
```

4.2. CRUD F2 `GetAsset(p_asset_id)`

Funkce vrátí všechny informace o assetu (název, typ, hlavní autor, licence, formát, velikost, aktivní stav, popis). Jako parametr musí být poskytnut jednoznačný identifikátor assetu.

Funkce také zjistí, která schválená verze je pro daný asset aktuálně k dispozici.

```
select a.asset_id, a.name, at.name as asset_type, au.name as author_name,
       l.name as license_name, a.file_format, a.size_mb, a.created_at,
       a.is_active, a.description
from Asset a
join AssetType at on at.asset_type_id = a.asset_type_id
join Author au on au.author_id = a.main_author_id
join License l on l.license_id = a.license_id
where a.asset_id = p_asset_id

select max(version_number) from AssetVersion
where asset_id = p_asset_id
and is_approved = TRUE
```

4.3. F3 `GetAssetVersions(p_asset_id)`

Funkce vrátí všechny verze spojené s assetem s identifikátorem `p_asset_id`.

```
select * from AssetVersion
where asset_id = p_asset_id
order by version_number desc
```

4.4. F4 `GetAssetProjects(p_asset_id)`

Funkce vrátí všechny projekty, ve kterých je zadaný asset použit.

```
select * from AssetProjectUsage apu
join Project p on p.project_id = apu.project_id
where apu.asset_id = p_asset_id
```

4.5. F5 `AddAsset(p_asset_id, p_name, p_asset_type_id, p_main_author_id, p_license_id, p_file_format, p_size_mb, p_created_at, p_is_active, p_description)`

Transakce vloží nový asset do tabulky `Asset`. Před vložením ověřuje, že zadaný typ assetu, hlavní autor a licence v systému existují. Po úspěšném vložení je asset připraven k dalšímu verzování a přiřazení do projektů.

```
insert into Asset(asset_id, name, asset_type_id, main_author_id, license_id,
                 file_format, size_mb, created_at, is_active, description)
values(p_asset_id, p_name, p_asset_type_id, p_main_author_id, p_license_id,
       p_file_format, p_size_mb, p_created_at, p_is_active, p_description)
```

4.6. T1 F6 `AddAssetVersion(p_asset_version_id, p_asset_id, p_version_number, p_file_path, p_created_at, p_changelog, p_is_approved)`

Transakce vloží záznam do tabulky `AssetVersion`. Pokud je nová verze označena jako schválená, všechny ostatní verze stejného assetu se automaticky nastaví jako neschválené, aby byla v systému vždy právě jedna schválená verze.

```
create or replace procedure AddAssetVersion(
  p_asset_version_id integer,
  p_asset_id integer,
  p_version_number integer,
  p_file_path varchar2,
  p_created_at date,
  p_changelog varchar2,
  p_is_approved number
) is
begin
  if p_is_approved = 1 then
    update AssetVersion
    set is_approved = 0
    where asset_id = p_asset_id;
  end if;

  insert into AssetVersion(
    asset_version_id, asset_id, version_number, file_path,
    created_at, changelog, is_approved
  )
  values (
    p_asset_version_id, p_asset_id, p_version_number, p_file_path,
    p_created_at, p_changelog, p_is_approved
  );
end;
```

4.7. T2 F7 `AddAssetProjectUsage(p_asset_id, p_project_id, p_usage_role, p_added_at, p_is_active)`

Transakce vloží záznam do tabulky `AssetProjectUsage`. Před vložením kontroluje, zda licence přiřazeného assetu dovoluje použití v daném projektu. Pokud je projekt ve stavu `released`, může být použit pouze asset s komerčně použitelnou licencí.

```
create or replace procedure AddAssetProjectUsage(
  p_asset_id integer,
  p_project_id integer,
  p_usage_role varchar2,
  p_added_at date,
  p_is_active number
) is
  v_is_commercial number;
  v_status Project.status%type;
  v_invalid_license exception;
begin
  select l.is_commercial, p.status
  into v_is_commercial, v_status
  from Asset a
  join License l on l.license_id = a.license_id
  join Project p on p.project_id = p_project_id
  where a.asset_id = p_asset_id;

  if v_status = 'released' and v_is_commercial = 0 then
    raise v_invalid_license;
  end if;
end;
```



```

end if;

insert into AssetProjectUsage(asset_id, project_id, usage_role, added_at, is_active)
values(p_asset_id, p_project_id, p_usage_role, p_added_at, p_is_active);
exception
when v_invalid_license then
    dbms_output.put_line('Licence assetu neumožňuje použití ve vydaném projektu.');
```

4.8. CRUD

F8 EditAsset(p_asset_id, p_name, p_license_id, p_file_format, p_size_mb, p_is_active, p_description)

Funkce aktualizuje záznam assetu na základě poskytnutého parametru `p_asset_id`.

```

update Asset
set name = p_name,
    license_id = p_license_id,
    file_format = p_file_format,
    size_mb = p_size_mb,
    is_active = p_is_active,
    description = p_description
where asset_id = p_asset_id
```

4.9. CRUD F9 FindAssets(p_asset_type_id, p_is_commercial)

Funkce vrátí seznam aktivních assetů, které odpovídají zadanému typu a zvolenému licenčnímu omezení.

```

select a.* from Asset a
join License l on l.license_id = a.license_id
where a.asset_type_id = p_asset_type_id
    and a.is_active = TRUE
    and l.is_commercial = p_is_commercial
```

4.10. F10 GetStats()

Funkce vrátí prvních deset assetů s nejvyšším počtem použití v projektech.

```

select a.asset_id, a.name, count(*) as usage_count
from AssetProjectUsage apu
join Asset a on a.asset_id = apu.asset_id
where apu.is_active = TRUE
group by a.asset_id, a.name
order by usage_count desc
fetch first 10 rows only
```

5. Popis tabulek databáze

5.1. Přehled tabulek

TABULKA	POPIS
Author	Autoři (tvůrci assetů, např. 3D artist, sound designer).
AssetType	Typy assetů (3D model, textura, zvuk, UI ikona, ...).
License	Licenční podmínky použití assetů (MIT, CC-BY, interní, ...).
Project	Herní projekty, ve kterých se assety používají.
Asset	Logické assety (např. konkrétní model či textura) bez ohledu na verzi.
AssetVersion	Konkrétní verze jednotlivých assetů (v1, v2, optimalizace, ...).
AssetProjectUsage	Použití assetů v projektech (M:N vazba Asset ↔ Project).

5.2. Tabulka Author

Tabulka Author eviduje tvůrce assetů (grafiky, zvukaře, programátory, ...).

SLOUPEC	TYP	POVINNÝ	POPIS
author_id	int	ano	Primární klíč autora.
name	varchar(100)	ano	Jméno a příjmení autora.
email	varchar(150)	ne	Kontaktní e-mail autora.
role	varchar(50)	ne	Role autora, např. „3D Artist“, „Sound Designer“.
created_at	date	ano	Datum založení záznamu autora v systému.

5.3. Tabulka AssetType

Tabulka AssetType popisuje typy assetů (modely, textury, zvuky, ...).

SLOUPEC	TYP	POVINNÝ	POPIS
asset_type_id	int	ano	Primární klíč typu assetu.
name	varchar(50)	ano	Název typu assetu, např. „3D model“, „Texture“, „Sound“.
description	varchar(255)	ne	Podrobnější textový popis typu assetu.

5.4. Tabulka License

Tabulka License obsahuje licenční podmínky jednotlivých assetů.

SLOUPEC	TYP	POVINNÝ	POPIS
license_id	int	ano	Primární klíč licence.

name	varchar(100)	ano	Název licence, např. „MIT“, „CC-BY 4.0“, „Internal“.
license_url	varchar(255)	ne	Odkaz na plné znění licence (pokud existuje).
is_commercial	boolean	ano	Informace, zda je licence použitelná v komerčních projektech.
usage_notes	varchar(500)	ne	Stručné shrnutí podmínek použití nebo omezení licence.

5.5. Tabulka `Project`

Tabulka `Project` reprezentuje herní projekty, ve kterých jsou assety použity.

SLOUPEC	TYP	POVINNÝ	POPIS
project_id	int	ano	Primární klíč projektu.
name	varchar(100)	ano	Název herního projektu.
description	varchar(500)	ne	Stručný popis projektu.
status	varchar(30)	ne	Stav projektu, např. „in_development“, „released“, „planning“.
start_date	date	ne	Datum začátku vývoje projektu.
end_date	date	ne	Datum ukončení vývoje projektu (pokud je k dispozici).

5.6. Tabulka `Asset`

Tabulka `Asset` reprezentuje logické assety (např. model „Warrior_Model“), bez ohledu na konkrétní verzi souboru.

SLOUPEC	TYP	POVINNÝ	POPIS
asset_id	int	ano	Primární klíč assetu.
name	varchar(100)	ano	Název assetu, např. „Warrior_Model“.
asset_type_id	int	ano	Cizí klíč na <code>AssetType</code> , určuje typ assetu.
main_author_id	int	ano	Cizí klíč na <code>Author</code> , hlavní autor assetu.
license_id	int	ano	Cizí klíč na <code>License</code> , určuje licenci assetu.
file_format	varchar(10)	ano	Formát souboru, např. „fbx“, „png“, „wav“.
size_mb	decimal(10,2)	ne	Velikost assetu v megabajtech.
created_at	date	ano	Datum založení assetu v systému.
is_active	boolean	ano	Indikace, zda je asset aktivně používán.
description	varchar(500)	ne	Textový popis assetu, účel, poznámky.

5.7. Tabulka `AssetVersion`

Tabulka `AssetVersion` reprezentuje jednotlivé verze assetů (v1, v2, optimalizovaná verze, ...).

SLOUPEC	TYP	POVINNÝ	POPIS
<code>asset_version_id</code>	int	ano	Primární klíč verze assetu.
<code>asset_id</code>	int	ano	Cizí klíč na <code>Asset</code> , ke kterému verze patří.
<code>version_number</code>	int	ano	Číselné označení verze (1, 2, 3, ...).
<code>file_path</code>	varchar(255)	ano	Cesta k souboru verze assetu v úložišti.
<code>created_at</code>	date	ano	Datum vytvoření dané verze assetu.
<code>changelog</code>	varchar(500)	ne	Stručný popis změn oproti předchozí verzi.
<code>is_approved</code>	boolean	ano	Indikace, zda je verze schválená pro použití v projektech.

5.8. Tabulka `AssetProjectUsage`

Tabulka `AssetProjectUsage` popisuje použití assetů v konkrétních projektech (M:N vazba mezi `Asset` a `Project`).

SLOUPEC	TYP	POVINNÝ	POPIS
<code>asset_id</code>	int	ano	Cizí klíč na <code>Asset</code> . Součást složeného primárního klíče.
<code>project_id</code>	int	ano	Cizí klíč na <code>Project</code> . Součást složeného primárního klíče.
<code>usage_role</code>	varchar(100)	ne	Popis role assetu v projektu, např. „enemy_model“, „menu_background“.
<code>added_at</code>	date	ano	Datum přidání assetu do projektu.
<code>is_active</code>	boolean	ano	Indikace, zda je použití assetu v projektu stále aktivní.

6. Minispecifikace netriviální funkce

6.1. Vybraná transakce

Pro detailní minispecifikaci je zvolena funkce `T1 F6 AddAssetVersion`. Tato funkce je reálnou transakcí informačního systému, protože při schválení nové verze neprovádí pouze obyčejný insert do tabulky `AssetVersion`, ale zároveň musí změnit stav ostatních verzí stejného assetu. Všechny kroky musí proběhnout buď celé, nebo vůbec, aby v systému nevznikl nekonzistentní stav se dvěma současně schválenými verzemi téhož assetu.

6.2. Cíle transakce

1. ověřit, že cílový asset existuje a lze k němu přidat novou verzi,
2. ověřit, že číslo verze není pro daný asset duplicitní,
3. při schválení nové verze odebrat schválení všem ostatním verzím stejného assetu,
4. vložit novou verzi assetu se zadanými metadaty,
5. zachovat konzistentní stav, ve kterém má asset po dokončení transakce nejvýše jednu schválenou verzi.

6.3. Vstupy a výstupy

- Vstupy: `p_asset_version_id`, `p_asset_id`, `p_version_number`, `p_file_path`, `p_created_at`, `p_changelog`, `p_is_approved`.
- Výstup: logická hodnota `true` při úspěšném vložení nové verze, jinak `false`.

6.4. Minispecifikace krok za krokem

1. `start transaction;`
2. Načti cílový asset podle `p_asset_id` a uzamkni jej proti souběžné změně.

```
select a.asset_id
into v_asset_id
from Asset a
where a.asset_id = p_asset_id
for update;
```

3. Ověř, že nová verze obsahuje povinné údaje a že číslo verze ještě pro daný asset neexistuje. Při neúspěchu proved' `rollback` a vrať `false`.

```
if p_file_path is null or p_version_number is null or p_version_number <= 0 then
    rollback;
    return false;
end if;
```

```
select count(*)
into v_version_exists
from AssetVersion av
where av.asset_id = p_asset_id
    and av.version_number = p_version_number;
```

```
if v_version_exists > 0 then
    rollback;
    return false;
end if;
```

4. Pokud má být nová verze schválená, načti a uzamkni existující schválené verze stejného assetu.

```
if p_is_approved = 1 then
  for r in (
    select av.asset_version_id
    from AssetVersion av
    where av.asset_id = p_asset_id
    and av.is_approved = 1
    for update
  ) loop
    null;
  end loop;
end if;
```

5. Pokud `p_is_approved = 1`, odeber schválení všem ostatním verzím stejného assetu.

```
if p_is_approved = 1 then
  update AssetVersion
  set is_approved = 0
  where asset_id = p_asset_id
  and is_approved = 1;
end if;
```

6. Vlož novou verzi do tabulky `AssetVersion`.

```
insert into AssetVersion(
  asset_version_id,
  asset_id,
  version_number,
  file_path,
  created_at,
  changelog,
  is_approved
) values (
  p_asset_version_id,
  p_asset_id,
  p_version_number,
  p_file_path,
  p_created_at,
  p_changelog,
  p_is_approved
);
```

7. Ověř výsledný stav schválení. Asset nesmí mít po dokončení transakce více než jednu schválenou verzi. Při porušení podmínky proved' `rollback` a vrať `false`.

```
select count(*)
into v_approved_count
from AssetVersion av
where av.asset_id = p_asset_id
and av.is_approved = 1;

if v_approved_count > 1 then
  rollback;
  return false;
end if;
```

8. `commit; return true;`

9. Při jakékoliv chybě proved' `rollback; return false; .`

6.5. Poznámky k transakci

- Transakce vyžaduje izolaci mezi načtením existujících verzí a jejich aktualizací. Proto je v minispecifikaci použito `for update` , aby dva uživatelé nemohli souběžně schválit dvě různé verze téhož assetu.
- Nestačí pouze zobrazit stav schválení ve formuláři. Mezi načtením detailu assetu a uložením nové verze může jiná transakce schválit jinou verzi.
- Funkce `AddAssetVersion` není triviální CRUD operace. Jde o transakční scénář, který pracuje s více řádky téže entity a zachovává invariant databáze.
- Pokud je nová verze vložena s `p_is_approved = 0` , transakce pouze přidá další verzi a ponechá dosavadní schválenou verzi beze změny.

7. Závěr

Navržený formulář pro správu assetů splňuje požadavky na funkční analýzu i detailní popis netriviální funkce. Pracuje s více tabulkami a vazbami, pokrývá běžné čtecí i zapisovací operace nad assety, verzemi a jejich použitím v projektech a obsahuje deset funkcí datové vrstvy vyvolávaných z navrženého rozhraní.

Jádrum návrhu je transakce `AddAssetVersion`, která jednoznačně ukazuje potřebu atomického zpracování a izolace při práci s databází. Zvolené řešení tak odpovídá reálnému informačnímu systému pro herní vývoj a současně zůstává dostatečně malé a přehledné pro semestrální projekt.

Index of Figures

Obrázek 1	Relační datový model systému správy assetů.	4
Obrázek 2	Vygenerovaný návrh hlavní obrazovky systému správy assetů.	5